МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №2

"Структури даних та функціональна декомпозиція"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

проф. Свелеба С. А.

Львів-2026

Мета роботи: Опанувати практичні навички професійного використання базових (list, tuple, set, dict) та розширених (collections: Counter, defaultdict) структур даних Python, генераторних виразів та comprehensions, навчитися оцінювати часову та просторову складність операцій, застосовувати функціональну декомпозицію, позиційні та іменовані аргументи (*args, **kwargs), lambda-функції, замикання (closures) та параметризовані декоратори. Індивідуальний варіант 1: система обліку студентів.

Теоретичні відомості:

Структури даних визначають організацію, зберігання та швидкість доступу до інформації в пам'яті. У Python list є впорядкованою змінною послідовністю (доступ за індексом O(1), лінійний пошук O(n)). tuple - впорядкована незмінна структура з низькими накладними витратами пам'яті. set зберігає унікальні хешовні елементи та забезпечує перевірку належності в середньому за O(1). dict реалізує хеш-таблицю ключ-значення з середньою складністю доступу за ключем O(1) та найгіршою O(n) при колізіях. Модуль collections надає розширені типи: Counter для підрахунку частот появи елементів та defaultdict для автоматичного створення значень за замовчуванням під час групування. List, set та dict comprehensions забезпечують компактний декларативний синтаксис побудови колекцій, а генераторні вирази (generator expressions) обчислюють агреговані значення потоково без виділення пам'яті під проміжні списки. Функціональна декомпозиція розбиває задачу на чисті функції з єдиною відповідальністю. Механізм *args дозволяє приймати змінну кількість позиційних аргументів, **kwargs - довільні іменовані параметри. Lambda створює анонімні функції, closure зберігає доступ до лексичного оточення зовнішньої функції, а декоратори з @wraps розширюють поведінку функцій зі збереженням їхніх метаданих.

Виконання роботи

1. Постановка індивідуального завдання (Варіант 1: Аналіз системи обліку студентів). Вхідні дані моделі містять: ім'я та прізвище, академічну групу та середній бал. Відповідно до варіанта реалізував та перевірив наступні операції: формування списку всіх студентів, визначення множини унікальних груп, розрахунок середнього бала всіх студентів, пошук студента з найвищим балом, фільтрацію студентів із балом строго вище заданого порогу, групування студентів за групами, формування рейтингу успішності, підрахунок кількості студентів у кожній групі та побудову словникового індексу за прізвищем зі збереженням однофамільців.

2. Обґрунтування вибору базових і спеціалізованих структур даних Python:
- list: використано для базової колекції students (Student) як впорядкованої змінюваної послідовності. Забезпечує послідовне збереження порядку

додавання, доступ за числовим індексом за O(1) та ефективне сортування;
- tuple: використано в модулі експериментів для генерації тестових записів (number, Student(...)) та передачі розмірів вибірок sizes = (1000, 10000, 100000). Незмінність кортежу гарантує цілісність конфігурації й мінімізує витрати пам'яті інтерпретатора;
- set: використано у функції unique_groups для збереження множини неповторюваних назв груп. Хешування рядків дозволяє вилучати дублікати за O(n) та виконувати перевірку належності за O(1);
- dict: використано для створення словникового індексу surname_index за прізвищем. Ключем виступає нормалізоване прізвище student.last_name.casefold(), а значенням є список знайдених студентів (list[Student]). Це дозволяє зберегти однофамільців (duplicate surnames) без втрати даних та виконувати швидкий пошук за O(1);
- Counter: застосовано в analytics.summarize для автоматичного підрахунку розподілу студентів за групами;
- defaultdict(list): застосовано в group_students та surname_index для групування елементів за ключем без громіздких перевірок наявності ключа у словнику.

3. Структура проєкту та організація модулів. Для підтримки чистої модульної архітектури зберіг src-layout. Розподіл обов'язків між файлами пакета student_manager наступний:
- models.py: датаклас Student із валідацією полів та властивістю full_name;
- analytics.py: чисті функції аналізу, фільтрації, сортування, групування, comprehensions та замикання;
- decorators.py: параметризований декоратор вимірювання часу measure_time зі збереженням метаданих через @wraps;
- demos.py: сценарій run_lab2 для комплексного запуску аналітики над тестовим набором;
- experiments.py: модуль вимірювання швидкодії лінійного пошуку в list проти пошуку в dict на вибірках до 100 000 записів;
- main.py: консольна точка входу з підтримкою прапорців --demo та --lab2;
- tests/: набір модульних тестів (test_analytics.py, test_experiments.py, test_lab_cli.py).

4. Датаклас Student у models.py: забезпечує строгу типізацію та валідацію вхідних значень. У методі __post_init__ видаляю зайві пробіли, блокую порожні рядки та контролюю діапазон середнього бала від 0 до 100. Властивість full_name формує повне ім'я студента з поверненням результату через форматований рядок.

```python
from dataclasses import dataclass


@dataclass
class Student:
    first_name: str
    last_name: str
    group: str
    average_grade: float
    student_id: int | None = None

    def __post_init__(self) -> None:
        self.first_name = self.first_name.strip()
        self.last_name = self.last_name.strip()
        self.group = self.group.strip()
        if not self.first_name or not self.last_name or not self.group:
            raise ValueError("Name, surname and group cannot be empty")
        if not 0 <= self.average_grade <= 100:
            raise ValueError("Average grade must be between 0 and 100")
        if self.student_id is not None:
            # перевіряємо шоб айді був цілим додатнім числом і не bool
            if isinstance(self.student_id, bool) or not isinstance(
                self.student_id, int
            ):
                raise TypeError("Student ID must be an integer")
            if self.student_id <= 0:
                raise ValueError("Student ID must be a positive integer")

    @property
    def full_name(self) -> str:
        return f"{self.first_name} {self.last_name}"
```

5. Функції обробки даних у analytics.py (частина 1). Реалізував set comprehension у функції unique_groups, обробку змінної кількості позиційних аргументів у функції average_values (*values), створення словникового запису через **kwargs у create_record, універсальну функцію фільтрації вищого порядку filter_students (із list comprehension) та замикання (closure) grade_filter.

```python
from collections import Counter, defaultdict
from collections.abc import Callable, Sequence

from student_manager.models import Student


def unique_groups(students: Sequence[Student]) -> set[str]:
    return {student.group for student in students}


def average_values(*values: float) -> float | None:
    return sum(values) / len(values) if values else None


def create_record(**fields: object) -> dict[str, object]:
    return dict(fields)


def filter_students(
    students: Sequence[Student], predicate: Callable[[Student], bool]
) -> list[Student]:
    return [student for student in students if predicate(student)]


def grade_filter(threshold: float) -> Callable[[Student], bool]:
    if not 0 <= threshold <= 100:
        raise ValueError("Threshold must be between 0 and 100")

    def predicate(student: Student) -> bool:
        return student.average_grade > threshold

    return predicate
```

Важливо відзначити реалізацію замикання: функція grade_filter(threshold) приймає пороговий бал і повертає предикат predicate(student). Внутрішня функція зберігає значення threshold у замиканні (__closure__) і виконує строге порівняння: student.average_grade > threshold. Студенти, які мають бал рівно на межі порогу, у вибірку не потрапляють, що строго відповідає постановці варіанта.

6. Функції сортування, групування та індексації в analytics.py (частина 2). Реалізував універсальне сортування sort_students з lambda-виразом за замовчуванням (key = lambda student: student.average_grade, reverse=True), групування за групами group_students за допомогою defaultdict(list), та словниковий індекс surname_index, який нормалізує ключ методом casefold() і використовує dict comprehension.

```python
def above_grade(students: Sequence[Student], threshold: float) -> list[Student]:
    return filter_students(students, grade_filter(threshold))


def sort_students(
    students: Sequence[Student],
    key: Callable[[Student], float | str] = lambda student: student.average_grade,
    *,
    reverse: bool = True,
) -> list[Student]:
    return sorted(students, key=key, reverse=reverse)


def group_students(students: Sequence[Student]) -> dict[str, list[Student]]:
    groups: defaultdict[str, list[Student]] = defaultdict(list)
    for student in students:
        groups[student.group].append(student)
    return dict(groups)


def surname_index(students: Sequence[Student]) -> dict[str, list[Student]]:
    names: defaultdict[str, list[Student]] = defaultdict(list)
    for student in students:
        names[student.last_name.casefold()].append(student)
    return {name: members for name, members in names.items()}
```

7. Зведена аналітика у функції summarize в analytics.py (частина 3). Функція агрегує всі показники в один підсумковий словник. Розрахунок загального середнього бала реалізовано за допомогою генераторного виразу (generator expression) sum(student.average_grade for student in students) / len(students), що дозволяє уникнути виділення пам'яті під проміжний список. Найкращий студент знаходиться через max із lambda-ключем, а розподіл за групами рахується структурою Counter.

```python
def summarize(students: Sequence[Student]) -> dict[str, object]:
    return {
        "groups": unique_groups(students),
        "average": (
            sum(student.average_grade for student in students) / len(students)
            if students
            else None
        ),
        "best": max(students, key=lambda student: student.average_grade, default=None),
        "counts": Counter(student.group for student in students),
        "ranking": sort_students(students),
        "grouped": group_students(students),
    }
```

8. Параметризований декоратор вимірювання часу measure_time у decorators.py. Декоратор приймає функцію зворотного виклику record: Callable[[float], None].

Внутрішня функція decorate загортає цільову функцію за допомогою @wraps(function), фіксує точний час початку через perf_counter() і в блоці finally передає інтервал виконання у колбек record. Завдяки цьому результати вимірювання можна спрямувати у список, логер чи зовнішнє сховище без модифікації логіки цільової функції.

```python
from collections.abc import Callable
from functools import wraps
from time import perf_counter
from typing import ParamSpec, TypeVar

P = ParamSpec("P")
R = TypeVar("R")


def measure_time(
    record: Callable[[float], None],
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    def decorate(function: Callable[P, R]) -> Callable[P, R]:
        @wraps(function)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            start = perf_counter()
            try:
                return function(*args, **kwargs)
            finally:
                record(perf_counter() - start)

        return wrapper

    return decorate
```

9. Демонстраційний сценарій run_lab2 у demos.py та інтеграція в консольний інтерфейс main.py. Функція run_lab2 викликає аналітичні методи над тестовим набором із 4 студентів і виводить у консоль результати всіх операцій варіанта 1. Час виконання функції summarize вимірюється параметризованим декоратором measure_time(timings.append). У main.py додано аргумент командного рядка --lab2.

```python
from student_manager import analytics
from student_manager.decorators import measure_time
from student_manager.models import Student


def run_lab2(students: list[Student]) -> None:
    timings: list[float] = []
    summary = measure_time(timings.append)(analytics.summarize)(students)
    print("Унікальні групи:", sorted(analytics.unique_groups(students)))
    print("Середній бал:", summary["average"])
    print("Найкращий студент:", summary["best"])
    print("Кількість за групами:", summary["counts"])
    print("Групування:", summary["grouped"])
    print("Рейтинг:")
    for student in analytics.sort_students(students):
        print(student.full_name, student.group, student.average_grade)
    print("Бал строго вище 90:", analytics.above_grade(students, 90))
    print("Пошук Коваль:", analytics.surname_index(students).get("коваль", []))
    print("*args:", analytics.average_values(80, 90, 100))
    print("**kwargs:", analytics.create_record(group="ФеП-33", count=3))
    print("Час аналітики, с:", timings[0])
```

```python
def main() -> None:
    parser = argparse.ArgumentParser(description="Система обліку студентів")
    parser.add_argument("--demo", action="store_true", help="показати приклад роботи")
    parser.add_argument(
        "--lab2", action="store_true", help="показати аналітику колекцій"
    )
    parser.add_argument("--lab4", action="store_true", help="показати об'єктну модель")
    parser.add_argument("--lab3", metavar="CSV", help="потокова обробка CSV")
    parser.add_argument("--config", metavar="YAML", help="надійний імпорт та експорт")
    args = parser.parse_args()
    if args.lab2:
        from student_manager.demos import run_lab2

        run_lab2(create_demo_students())
    elif args.lab4:
        from student_manager.lab4_demo import run_demo as lab4_main

        lab4_main()
    elif args.lab3:
        from student_manager.lab3_demo import main as lab3_main
```

Виконав запуск консольної команди з прапорцем --lab2. Програма успішно вивела унікальні групи, середній бал (89.85), найкращу студентку (Ірина Бондар, 95.2), розподіл груп Counter, словник групування, рейтинг студентів, вибірку з балом строго вище 90, пошук за прізвищем Коваль, роботу *args (середнє 90.0), **kwargs і тривалість виконання аналітики.

```
PS F:\university\3yr-1\professional_python> .\.venv\Scripts\python.exe -m student_manager.main --lab2
Унікальні групи: ['ФеП-32', 'ФеП-33']
Середній бал: 89.85
Найкращий студент: Student(first_name='Ірина', last_name='Бондар', group='ФеП-32', average_grade=95.2,
student_id=None)
Кількість за групами: Counter({'ФеП-33': 3, 'ФеП-32': 1})
Групування: {'ФеП-33': [Student(first_name='Олена', last_name='Коваль', group='ФеП-33', average_grade=91.5,
student_id=None), Student(first_name='Іван', last_name='Мельник', group='ФеП-33', average_grade=84.0,
student_id=None), Student(first_name='Андрій', last_name='Шевченко', group='ФеП-33', average_grade=88.7,
student_id=None)], 'ФеП-32': [Student(first_name='Ірина', last_name='Бондар', group='ФеП-32',
average_grade=95.2, student_id=None)]}
Рейтинг:
Ірина Бондар ФеП-32 95.2
Олена Коваль ФеП-33 91.5
Андрій Шевченко ФеП-33 88.7
Іван Мельник ФеП-33 84.0
Бал строго вище 90: [Student(first_name='Олена', last_name='Коваль', group='ФеП-33', average_grade=91.5,
student_id=None), Student(first_name='Ірина', last_name='Бондар', group='ФеП-32', average_grade=95.2,
student_id=None)]
Пошук Коваль: [Student(first_name='Олена', last_name='Коваль', group='ФеП-33', average_grade=91.5,
student_id=None)]
*args: 90.0
**kwargs: {'group': 'ФеП-33', 'count': 3}
Час аналітики, с: 7.399999958579428e-05
PS F:\university\3yr-1\professional_python> _
```

10. Локальний перевірочний харнес evidence_harness.py для поглибленого тестування та перевірки граничних випадків. Для уникнення надмірного перенесення довгих рядків dataclass у консолі та для прямої верифікації надійності створив автономний модуль перевірки. Він тестує дві критичні вимоги варіанта 1:

1) Збереження дублікатів у surname_index: додано двох різних студентів з однаковим прізвищем Коваль (Олена Коваль, ФеП-33 та Тарас Коваль, ФеП-31). Перевіряється, що індекс повертає список з обома записами, а не перезаписує попередній;

2) Строге відсікання замикання grade_filter(90.0): студент Тарас Коваль має бал рівно 90.0. Перевіряється, що предикат зі строгим знаком > виключає його з результуючого списку (потрапляють лише студенти з балами 91.5 та 95.2).

```python
"""Harness for capturing verified Lab 2 execution evidence."""

from collections.abc import Callable
import json
from student_manager import analytics
from student_manager.decorators import measure_time
from student_manager.experiments import lookup_benchmark
from student_manager.models import Student


def run_evidence() -> dict[str, object]:
    students = [
        Student("Олена", "Коваль", "ФеП-33", 91.5),
        Student("Іван", "Мельник", "ФеП-33", 84.0),
        Student("Ірина", "Бондар", "ФеП-32", 95.2),
        Student("Андрій", "Шевченко", "ФеП-33", 88.7),
        Student("Тарас", "Коваль", "ФеП-31", 90.0),
    ]

    timings: list[float] = []

    @measure_time(timings.append)
    def compute_summary(data: list[Student]) -> dict[str, object]:
        return analytics.summarize(data)

    summary = compute_summary(students)

    # 1. базові структури та операції
    groups = analytics.unique_groups(students)
    avg_grade = summary["average"]
    best_student = summary["best"]
    counts = summary["counts"]
    grouped = analytics.group_students(students)
    ranked = analytics.sort_students(students)
```

```python
    # 2. args та kwargs
    avg_args = analytics.average_values(85.0, 92.5, 78.0, 96.5)
    rec_kwargs = analytics.create_record(
        system="student_manager", lab=2, active=True, variant=1
    )

    # 3. перевірка надійності: дублікати прізвищ
    index = analytics.surname_index(students)
    koval_matches = index.get("коваль", [])

    # 4. перевірка надійності: строге порівняння у closure (> замість >=)
    filter_90 = analytics.grade_filter(90.0)
    strict_above_90 = analytics.filter_students(students, filter_90)
    excluded_exact_90 = [s for s in students if s.average_grade == 90.0]

    # 5. запуск бенчмарку
    bench_data = lookup_benchmark((1000, 10000, 100000), repeats=200)

    output = {
        "unique_groups": sorted(list(groups)),
        "average_grade": avg_grade,
        "best_student": {
            "name": best_student.full_name,
            "group": best_student.group,
            "grade": best_student.average_grade,
        }
        if best_student
        else None,
        "group_counts": dict(counts),
        "grouped_by_group": {
            g: [s.full_name for s in s_list] for g, s_list in grouped.items()
        },
        "ranking": [
```

```
    print("Групування за групами:")
    for grp, names in data["grouped_by_group"].items():
        print(f"  {grp}: {', '.join(names)}")
    print("Рейтинг студентів:")
    for s in data["ranking"]:
        print(f"  {s['name']:<20} {s['group']:<8} {s['grade']:>5.2f}")
    print(f"*args середнє: {data['args_result']:.2f}")
    print(f"**kwargs запис: {data['kwargs_result']}")
    print(f"Параметризований декоратор (час аналітики): {data['timing_seconds']:.6f} с")
    print("\n--- ПЕРЕВІРКА НАДІЙНОСТІ ТА МЕЖОВИХ СИТУАЦІЙ ---")
    dup = data["duplicate_surname_test"]
    print(
        f"Пошук однофамільців '{dup['query']}': знайдено {dup['matches_count']} студенти"
    )
    for s in dup["students"]:
        print(f"  {s['name']} ({s['group']}, {s['grade']})")
    print(f"  Збереження дублікатів у списку: {dup['preserved']}")

    thr = data["strict_threshold_test"]
    print(f"Строгий фільтр > {thr['threshold']}:")
    print(f"  Студент з балом рівно {thr['threshold']}: {thr['exact_90_student']}")
    print(f"  Чи потрапив бал рівно 90.0 у вибірку (> 90.0): {thr['exact_90_included']}")
    print("  Студенти строго вище 90.0:")
    for s in thr["strictly_above"]:
        print(f"    {s['name']}: {s['grade']}")


if __name__ == "__main__":
    main()
```

Результат виконання харнесу підтвердив коректність усіх операцій варіанта 1, збереження 2 однофамільців у списку за ключем 'коваль', а також строге виключення оцінки 90.0 з вибірки фільтра > 90.0.

```
PS F:\university\3ур-1\professional_python> .\.venv\Scripts\python.exe .local_workflow\lab2\evidence_harness.py
=== АНАЛІТИЧНИЙ ЗВІТ ХАРНЕСУ LAB 2 ===
Унікальні групи: ['ФеП-31', 'ФеП-32', 'ФеП-33']
Загальний середній бал: 89.88
Найкращий студент: Ірина Бондар (ФеП-32) - 95.20
Кількість за групами: {'ФеП-33': 3, 'ФеП-32': 1, 'ФеП-31': 1}
Групування за групами:
  ФеП-33: Олена Коваль, Іван Мельник, Андрій Шевченко
  ФеП-32: Ірина Бондар
  ФеП-31: Тарас Коваль
Рейтинг студентів:
  Ірина Бондар         ФеП-32   95.20
  Олена Коваль         ФеП-33   91.50
  Тарас Коваль         ФеП-31   90.00
  Андрій Шевченко      ФеП-33   88.70
  Іван Мельник         ФеП-33   84.00
*args середнє: 88.00
**kwargs запис: {'system': 'student_manager', 'lab': 2, 'active': True, 'variant': 1}
Параметризований декоратор (час аналітики): 0.000131 с

--- ПЕРЕВІРКА НАДІЙНОСТІ ТА МЕЖОВИХ СИТУАЦІЙ ---
Пошук однофамільців 'коваль': знайдено 2 студенти
  Олена Коваль (ФеП-33, 91.5)
  Тарас Коваль (ФеП-31, 90.0)
  Збереження дублікатів у списку: True
Строгий фільтр > 90.0:
  Студент з балом рівно 90.0: Тарас Коваль
  Чи потрапив бал рівно 90.0 у вибірку (> 90.0): False
  Студенти строго вище 90.0:
    Олена Коваль: 91.5
    Ірина Бондар: 95.2
PS F:\university\3ур-1\professional_python> _
```

11. Експериментальне дослідження швидкодії: лінійний пошук у list проти пошуку за ключем у dict (experiments.py). У функції lookup_benchmark генерується синтетичний набір даних розміром 1 000, 10 000 та 100 000 записів у вигляді кортежів (number, Student(...)). Окремо вимірюється час побудови словникового індексу. Для чистоти експерименту шуканий елемент розміщено в самому кінці списку (target = size - 1), що моделює найгірший випадок для лінійного пошуку. Для нейтралізації похибки таймера кожен пошук

повторюється 200 разів, а середній час ділиться на кількість повторень. Час генерації даних виключено з вимірювань.

```python
import argparse
import json
from time import perf_counter

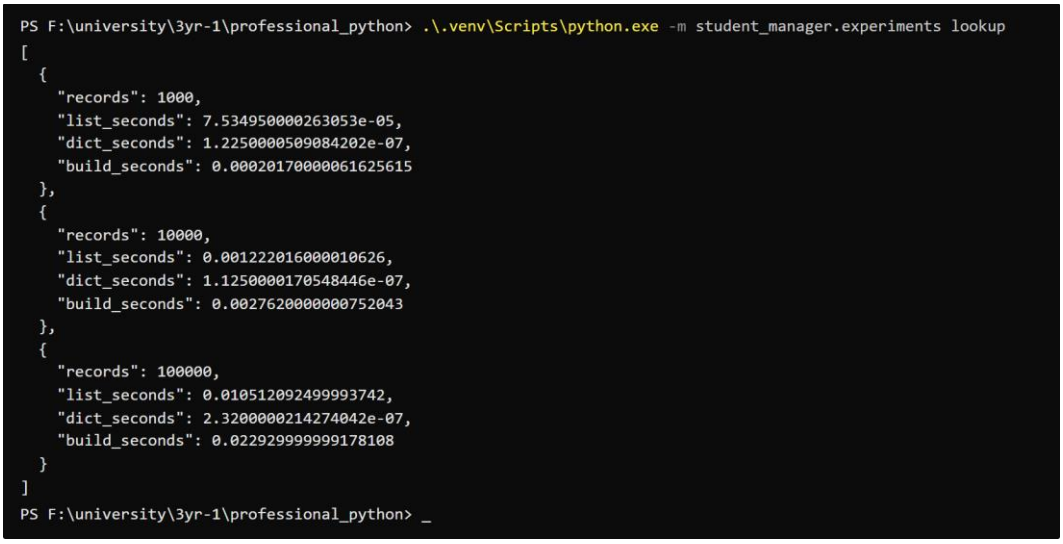
from student_manager.models import Student


def lookup_benchmark(
    sizes: tuple[int, ...] = (1000, 10000, 100000), *, repeats: int = 200
) -> list[dict[str, float | int]]:
    if repeats <= 0 or any(size <= 0 for size in sizes):
        raise ValueError("Sizes and repeats must be positive")
    results = []
    for size in sizes:
        records = [
            (number, Student("Student", str(number), "A", 80)) for number in range(size)
        ]
        start = perf_counter()
        index = {number: student for number, student in records}
        build = perf_counter() - start
        target = size - 1
        linear: Student | None = None
        indexed: Student | None = None
        start = perf_counter()
        for _ in range(repeats):
            linear = next(
                (student for number, student in records if number == target), None
            )
        linear_time = (perf_counter() - start) / repeats
        start = perf_counter()
        for _ in range(repeats):
            indexed = index.get(target)
        indexed_time = (perf_counter() - start) / repeats
        assert linear is indexed
        results.append(
            {
                "records": size,
                "list_seconds": linear_time,
                "dict_seconds": indexed_time,
                "build_seconds": build,
            }
        )
    return results


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("experiment", choices=["lookup"])
    parser.parse_args()
    print(json.dumps(lookup_benchmark(), indent=2))


if __name__ == "__main__":
    main()
```

Виконав запуск бенчмарку через командний рядок: python -m student_manager.experiments lookup. Отримані дані у форматі JSON зафіксовано нижче.



12. Теоретичний аналіз часової та просторової складності операцій. У таблиці нижче наведено порівняння характеристик основних структур даних, використаних у лабораторній роботі, з урахуванням компромісу між швидкістю доступу та споживанням оперативної пам'яті (Memory tradeoff).

Операція	**Структура**	**Середня складність**	**Найгірша складність**	**Особливості та пам'ять**
Доступ за індексом	list	O(1)	O(1)	Прямий доступ за зміщенням покажчика; мінімальні накладні витрати пам'яті.
Лінійний пошук	list	O(n)	O(n)	Повний послідовний перебір елементів списку; не потребує додаткової пам'яті.
Побудова індексу	dict (з list/tuple)	O(n) очікувана	$O(n^2)$	Одноразовий прохід з вставкою пар у хеш-таблицю; найгірший час $O(n^2)$ через масові колізії; вимагає O(n) пам'яті.
Пошук за ключем	dict	O(1)	O(n)	Хешування ключа дає прямий доступ; найгірший час при колізіях; пам'ять O(n).
Перевірка / додавання	set	O(1)	O(n)	Зберігає лише унікальні хешовні ключі; найгірший час при колізіях; пам'ять O(n).
Сортування (Timsort)	list	O(n log n)	O(n log n)	Адаптивний гібрид сортування злиттям і вставками; потребує O(n) пам'яті.
Компроміс пам'яті	dict / set vs list	O(n) пам'яті	O(n) пам'яті	dict/set резервують хеш-таблицю заради O(1) пошуку; накладні витрати пам'яті залежать від навантаження (workload-dependent) та коефіцієнта заповнення.

13. Зведена таблиця результатів практичного бенчмарку. Значення отримані на основі реального виконання experiments.py. На вибірці 100 000 записів

словниковий пошук демонструє стабільний час доступу близько 0.23 мікросекунди, тоді як лінійний пошук зростає до 10.51 мілісекунди.

Кількість записів (n)	Пошук у list (O(n)), с	Пошук у dict (O(1)), с	Співвідношення (list / dict)	Побудова індексу dict, с
1 000	0.00007535 (75.35 мкс)	0.00000012 (0.12 мкс)	615.1x	0.000202 (0.20 мс)
10 000	0.00122202 (1222.02 мкс)	0.00000011 (0.11 мкс)	10862.4x	0.002762 (2.76 мс)
100 000	0.01051209 (10512.09 мкс)	0.00000023 (0.23 мкс)	45310.7x	0.022930 (22.93 мс)

Аналіз практичних результатів бенчмарку:
- Лінійний пошук у list демонструє поведінку O(n) із системними коливаннями: час пошуку зростає при збільшенні вибірки (з 75.35 мкс до 10 512.09 мкс), що узгоджується з лінійною складністю з урахуванням шуму ОС та кеш-пам'яті процесора;
- Пошук за ключем у dict демонструє асимптотичну поведінку O(1): час доступу залишається практично незмінним (0.11-0.23 мкс) незалежно від розміру вибірки;
- Побудова словникового індексу має очікувану лінійну складність O(n) за часом і пам'яттю (0.20 мс для 1 000 записів та 22.93 мс для 100 000 записів), проте в найгіршому випадку масових колізій може деградувати до $O(n^2)$. Одноразова побудова індексу швидко окупається при регулярних повторних запитах;
- Реальні вимірювання містять системний шум ОС та вплив кеш-пам'яті процесора, тому абсолютне співвідношення коливається від ~615x до ~45 000x, що повністю підтверджує теоретичну перевагу хеш-таблиць для багаторазового пошуку.

14. Реалізація завдань підвищеної складності (>=3 розширень). У роботі реалізовано наступні розширені можливості з інструкції:
1) Універсальна функція фільтрації filter_students: приймає довільний функціональний предикат Callable[[Student], bool], що дозволяє гнучко фільтрувати студентів за будь-якими складеними умовами без дублювання циклів;
2) Універсальне сортування sort_students: функція вищого порядку підтримує передачу довільного key-екстрактора (Callable[[Student], float | str]) та напрямку reverse, забезпечуючи впорядкування за будь-яким полем моделі;
3) Параметризований декоратор вимірювання часу measure_time: підтримує передачу зовнішнього контейнера чи логера через аргумент record, зберігає сигнатуру цільової функції за допомогою @wraps та ізолює заміри часу від

основної бізнес-логіки;
4) Замикання grade_filter для генерації спеціалізованих фільтрів із параметром порогу.

15. Модульне тестування та перевірка синтаксису. Для забезпечення високої надійності розроблено набір автоматичних перевірок стандартною бібліотекою unittest. Тести покривають: збереження дублікатів однофамільців у surname_index, строге відсікання в grade_filter, збереження метаданих у measure_time, роботу *args та **kwargs, валідацію параметрів бенчмарку та консольний інтерфейс CLI.

```python
import importlib
import importlib.util
import unittest

from student_manager.models import Student


class AnalyticsTests(unittest.TestCase):
    def test_summary_keeps_duplicate_surnames_and_strict_threshold(self):
        self.assertIsNotNone(importlib.util.find_spec("student_manager.analytics"))
        analytics = importlib.import_module("student_manager.analytics")
        students = [
            Student("Anna", "Koval", "A", 90),
            Student("Oleh", "Koval", "B", 85),
            Student("Ira", "Bondar", "A", 95),
        ]
        summary = analytics.summarize(students)
        self.assertEqual(summary["groups"], {"A", "B"})
        self.assertEqual(summary["average"], 90)
        self.assertEqual(summary["counts"], {"A": 2, "B": 1})
        self.assertEqual(summary["ranking"], [students[2], students[0], students[1]])
        self.assertEqual(analytics.surname_index(students)["koval"], students[:2])
        self.assertEqual(analytics.above_grade(students, 90), [students[2]])
        self.assertIsNone(analytics.summarize([])["average"])
```

```python
    def test_timing_decorator_preserves_result_and_metadata(self):
        self.assertIsNotNone(importlib.util.find_spec("student_manager.decorators"))
        module = importlib.import_module("student_manager.decorators")
        measurements = []

        @module.measure_time(measurements.append)
        def total(*values, scale=1):
            return sum(values) * scale

        self.assertEqual(total(2, 3, scale=2), 10)
        self.assertEqual(total.__name__, "total")
        self.assertEqual(len(measurements), 1)
        self.assertGreaterEqual(measurements[0], 0)

    def test_functional_extensions(self):
        analytics = importlib.import_module("student_manager.analytics")
        students = [Student("A", "Z", "A", 80), Student("B", "Y", "B", 90)]
        self.assertEqual(analytics.average_values(80, 90), 85)
        self.assertIsNone(analytics.average_values())
        self.assertEqual(analytics.create_record(group="A"), {"group": "A"})
        self.assertEqual(
            analytics.filter_students(students, lambda s: s.group == "B"), students[1:]
        )
        self.assertEqual(
            analytics.sort_students(students, lambda s: s.last_name, reverse=False),
            students[::-1],
        )
        with self.assertRaises(ValueError):
            analytics.grade_filter(float("nan"))


if __name__ == "__main__":
    unittest.main()
```

```
PS F:\university\3yr-1\professional_python> .\.venv\Scripts\python.exe -m unittest discover -s tests -p
test_analytics.py
...
----------------------------------------------------------------------
Ran 3 tests in 0.020s

OK
PS F:\university\3yr-1\professional_python> _
```

```python
import importlib
import importlib.util
import unittest


class BenchmarkTests(unittest.TestCase):
    def test_invalid_measurement_parameters_are_rejected(self):
        module = importlib.import_module("student_manager.experiments")
        for sizes, repeats in [((0,), 1), ((-1,), 1), ((10,), 0)]:
            with (
                self.subTest(sizes=sizes, repeats=repeats),
                self.assertRaises(ValueError),
            ):
                module.lookup_benchmark(sizes, repeats=repeats)

    def test_lookup_measurements_use_equal_results(self):
        self.assertIsNotNone(importlib.util.find_spec("student_manager.experiments"))
        module = importlib.import_module("student_manager.experiments")
        rows = module.lookup_benchmark((10, 100), repeats=3)
        self.assertEqual([row["records"] for row in rows], [10, 100])
        for row in rows:
            self.assertGreaterEqual(row["list_seconds"], 0)
            self.assertGreaterEqual(row["dict_seconds"], 0)
            self.assertGreaterEqual(row["build_seconds"], 0)
```

```
PS F:\university\3yr-1\professional_python> .\.venv\Scripts\python.exe -m unittest discover -s tests -p
test_experiments.py
..
----------------------------------------------------------------------
Ran 2 tests in 0.240s

OK
PS F:\university\3yr-1\professional_python> _
```

Для перевірки консольного інтерфейсу другої лабораторної роботи викликав цільовий метод test_lab2_demo_shows_analytics класу LabCliTests із tests/test_lab_cli.py. Метод перевіряє виклик точки входу з аргументом --lab2, код повернення 0 та наявність ключових маркерів аналітики у вихідному потоці.

```python
    def test_lab2_demo_shows_analytics(self):
        result = subprocess.run(
            [sys.executable, "-m", "student_manager.main", "--lab2"],
            check=False,
            capture_output=True,
            encoding="utf-8",
            env={**os.environ, "PYTHONIOENCODING": "utf-8"},
            timeout=10,
        )
        self.assertEqual(result.returncode, 0, result.stderr)
        self.assertIn("Унікальні групи", result.stdout)
        self.assertIn("Рейтинг", result.stdout)
        self.assertIn("Коваль", result.stdout)
```

```
PS F:\university\3yr-1\professional_python> .\.venv\Scripts\python.exe -m unittest
tests.test_lab_cli.LabCliTests.test_lab2_demo_shows_analytics
.
----------------------------------------------------------------------
Ran 1 test in 0.594s

OK
PS F:\university\3yr-1\professional_python> _
```

16. Перевірка синтаксису та компіляції вихідних файлів лабораторної роботи. Виконав перевірку модулів analytics.py, decorators.py, demos.py та experiments.py утилітою py_compile. Синтаксичних помилок чи зауважень інтерпретатора не виявлено.

```
PS F:\university\3yr-1\professional_python> .\.venv\Scripts\python.exe -m py_compile
src\student_manager\analytics.py src\student_manager\decorators.py src\student_manager\demos.py
src\student_manager\experiments.py
PS F:\university\3yr-1\professional_python> _
```

Висновок:

Під час виконання лабораторної роботи №2 успішно розробив модуль обробки та аналізу даних системи обліку студентів відповідно до індивідуального варіанта 1. Застосував базові колекції (list, tuple, set, dict) та спеціалізовані структури модуля collections (Counter, defaultdict), обґрунтувавши доцільність кожного вибору. Реалізував усі три види comprehensions (list, set, dict) та генераторний вираз для обчислення середнього бала. Здійснив функціональну декомпозицію логіки на невеликі функції, реалізував підтримку *args, **kwargs, lambda-виразів, замикання зі строгим порівнянням порогу успішності та параметризованого декоратора вимірювання часу. Експериментально підтвердив теоретичні оцінки складності на вибірках до 100 000 записів: пошук за ключем у dict зберігає стабільний час O(1) (~0.23 мкс), тоді як пошук у list демонструє зростання O(n) до 10.51 мс, компенсуючи разові витрати часу на побудову індексу (O(n) очікувана) та додаткову пам'ять, обсяг якої залежить від робочого навантаження (workload-dependent). Усі операції варіанта та розширені можливості підтверджено автоматичними модульними тестами.